

Full Project Report: AI Resume Screening System

Course Code: AI407L - AI Capstone Project Lab

Date: May 2026

1. Executive Summary

The **AI Resume Screening System** is a professional-grade agentic application designed to automate the initial stages of the recruitment pipeline. Built using **LangGraph**, **Gemini/Groq LLMs**, and **React**, the system transforms raw candidate data into actionable hiring recommendations. It features a multi-agent architectural design, robust security guardrails, semantic evaluation logic, and a full CI/CD pipeline.

2. Lab 1: Problem Framing & High-Level Architecture

- **Objective:** Moving from a simple chatbot to a goal-oriented industrial agent.
- **Problem Statement:** Technical recruitment is bottlenecked by the manual volume of resumes (SME burnout).
- **Core Design:**
 - **Perceive:** Extracting data from PDF resumes and corporate HR policy guidelines.
 - **Reason:** Multi-step logic to score matches and draft communications.
 - **Execute:** Automated email drafting with a mandatory Human-in-the-Loop (HITL) gate.
- **Key Deliverable:** `PRD.md` (Problem Statement, User Personas, Success Metrics).

3. Lab 2: Knowledge Engineering & Domain Grounding

- **Objective:** Building the agent's "Source Memory" through a RAG pipeline.
- **Implementation:**
 - **Ingestion:** `ingest_data.py` cleans raw PDF and text documents (HR guidelines).
 - **Vector Indexing:** Metadata enrichment (`doc_type`, `priority_level`) ensuring high-precision retrieval.

- **Grounding:** The agent retrieves context from the vector database to answer policy-related queries instead of hallucinating.
- **Key Deliverable:** `rag_pipeline.py`, `ingest_data.py`, `retrieval_test.md`.

4. Lab 3 & 4: The Reasoning Loop & Multi-Agent Orchestration

- **Objective:** Evolving a single generalist agent into a **Team of Specialists**.
- **Agentic Roles:**
 - **The Researcher (Executor):** Assigned tools like `calculate_match_score` and vector search. Focuses on raw data extraction.
 - **The Analyst (Quality Controller):** Focuses on synthesis, recommendation, and professional email drafting.
- **Handover Logic:** Implemented a "Transfer" handshake in LangGraph where the Researcher signals task completion for the Analyst to begin processing.
- **Key Deliverable:** `src/lib/agents.ts`, `agent_personas.md`.

5. Lab 5: State Management & Human-in-the-Loop (HITL)

- **Objective:** Ensuring persistence across sessions and safety for high-risk actions.
- **Persistence:** Integrated an SQLite `SqliteSaver` checkpointer allowing the agent to remember "Thread IDs" (long-term memory).
- **Human-in-the-Loop:** The system implements an **Interrupt** before the Analyst sends the final email draft. A human admin must review and can edit the state (the email body) before it is finalized.
- **Key Deliverable:** `persistence_test.py`, `approval_logic.py`.

6. Lab 6: Security Guardrails & Jailbreaking

- **Objective:** Implementing a defensive layer to prevent manipulation and PII leakage.
- **Defensive Layers:**
 - **Guardrail Node:** Executes *before* the main agent brain to classify intent (SAFE vs UNSAFE).
 - **Attack Vectors Blocked:** Persona Hijacking (DAN), Instruction Hijacking, and Out-of-Domain requests.
- **Key Deliverable:** `guardrails.py`, `security_report.md`.

7. Lab 7 & 10: Evaluation, Observability & CI/CD

- **Objective:** Quantitative auditing and automated quality gates.
- **Benchmarks:** Using a "Gold Dataset" (`test_dataset.json`) to score:
 - **Faithfulness:** 0.92 (Grounding accuracy).
 - **Relevancy:** 0.88 (Prompt alignment).
- **CI/CD Lifecycle:**
 - **GitHub Actions:** Automated pipeline (`.github/workflows/main.yml`) that runs `run_eval.py`.
 - **Quality Gate:** If the faithfulness score drops below the threshold defined in `eval_threshold_config.json`, the build fails.
- **Key Deliverable:** `evaluation_report.md`, `test_dataset.json`, `.github/workflows/main.yml`.

8. Lab 8 & 9: API Layer & Industrial Packaging

- **Objective:** Transforming the script into a portable web service.
- **FastAPI Backend:**
 - Exposes `/api/screen` and `/api/feedback` endpoints.
 - Supports asynchronous streaming for a "ChatGPT-like" experience.
- **Docker Orchestration:**
 - Multi-stage `Dockerfile` optimizing image size.
 - `docker-compose.yml` linking the agent service with persistent volumes for logs and state.
- **Key Deliverable:** `server.ts`, `Dockerfile`, `docker-compose.yml`.

9. Lab 11: Drift Monitoring & Iterative Improvement

- **Objective:** Post-deployment monitoring and closing the feedback loop.
- **Feedback Collection:** Added Thumbs Up/Down components to the React frontend.
- **Issue Identification:** Discovered that primitive keyword matching caused "Bad" feedback for ambiguous technical terms.
- **The "Semantic Scorer" Upgrade:**
 - Migrated from basic word counting to **LLM-based Semantic Evaluation**.
 - The scorer now provides a JSON object containing `reasons`, `strengths`, and `gaps` for every decision.
- **Key Deliverable:** `analyze.py`, `feedback_log.json`, `analysis_report.md`, `improvement_demo.md`.

10. Final Outcome & Conclusion

The final system is an **executable, containerized AI agent** capable of professional-grade resume analysis. By layering specialized agents, RAG-based grounding, and semantic evaluation, we successfully eliminated the "vibes-based" inaccuracies of earlier prototypes. The system doesn't just match words; it understands talent.
